



Red Hat OpenShift AI Self-Managed 3.0

Evaluating AI systems

Evaluate your OpenShift AI models for accuracy, relevance, and consistency

Red Hat OpenShift AI Self-Managed 3.0 Evaluating AI systems

Evaluate your OpenShift AI models for accuracy, relevance, and consistency

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Evaluate your OpenShift AI models for accuracy, relevance, and consistency.

Table of Contents

CHAPTER 1. OVERVIEW OF EVALUATING AI SYSTEMS	3
CHAPTER 2. EVALUATING LARGE LANGUAGE MODELS	4
2.1. SETTING UP LM-EVAL	4
2.2. ENABLING EXTERNAL RESOURCE ACCESS FOR LMEVAL JOBS	5
2.2.1. Enabling online access and remote code execution for LMEval Jobs using the CLI	5
2.2.2. Updating LMEval job configuration using the web console	8
2.3. LM-EVAL EVALUATION JOB	9
2.4. LM-EVAL EVALUATION JOB PROPERTIES	11
2.4.1. Properties for setting up custom Unitxt cards, templates, or system prompts	15
2.5. PERFORMING MODEL EVALUATIONS IN THE DASHBOARD	16
2.6. LM-EVAL SCENARIOS	19
2.6.1. Accessing Hugging Face models with an environment variable token	19
2.6.2. Using a custom Unitxt card	20
2.6.3. Using PVCs as storage	22
2.6.3.1. Managed PVCs	22
2.6.3.2. Existing PVCs	23
2.6.4. Using a KServe Inference Service	23
2.6.5. Setting up LM-Eval S3 Support	25
2.6.6. Using LLM-as-a-Judge metrics with LM-Eval	27
CHAPTER 3. EVALUATING RAG SYSTEMS WITH RAGAS	33
3.1. ABOUT RAGAS EVALUATION	33
3.1.1. Key Ragas metrics	33
3.1.2. Use cases for Ragas in AI engineering workflows	34
3.1.3. Ragas provider deployment modes	34
3.2. SETTING UP THE RAGAS INLINE PROVIDER FOR DEVELOPMENT	35
3.3. CONFIGURING THE RAGAS REMOTE PROVIDER FOR PRODUCTION	37
3.4. EVALUATING RAG SYSTEM QUALITY WITH RAGAS METRICS	42
CHAPTER 4. USING LLAMA STACK WITH TRUSTYAI	45
4.1. USING LLAMA STACK EXTERNAL EVALUATION PROVIDER WITH LM-EVALUATION-HARNESS IN TRUSTYAI	45
4.2. RUNNING CUSTOM EVALUATIONS WITH LM-EVAL AND LLAMA STACK	48
4.3. DETECTING PERSONALLY IDENTIFIABLE INFORMATION (PII) BY USING GUARDRAILS WITH LLAMA STACK	51

CHAPTER 1. OVERVIEW OF EVALUATING AI SYSTEMS

Evaluate your AI systems to generate an analysis of your model's ability by using the following TrustyAI tools:

- **LM-Eval:** You can use TrustyAI to monitor your LLM against a range of different evaluation tasks and to ensure the accuracy and quality of its output. Features such as summarization, language toxicity, and question-answering accuracy are assessed to inform and improve your model parameters.
- **RAGAS:** Use Retrieval-Augmented Generation Assessment (RAGAS) with TrustyAI to measure and improve the quality of your RAG systems in OpenShift AI. RAGAS provides objective metrics that assess retrieval quality, answer relevance, and factual consistency.
- **Llama Stack:** Use Llama Stack components and providers with TrustyAI to evaluate and work with LLMs.

CHAPTER 2. EVALUATING LARGE LANGUAGE MODELS

A large language model (LLM) is a type of artificial intelligence (AI) program that is designed for natural language processing tasks, such as recognizing and generating text.

As a data scientist, you might want to monitor your large language models against a range of metrics, in order to ensure the accuracy and quality of its output. Features such as summarization, language toxicity, and question-answering accuracy can be assessed to inform and improve your model parameters.

Red Hat OpenShift AI now offers Language Model Evaluation as a Service (LM-Eval-aaS), in a feature called LM-Eval. LM-Eval provides a unified framework to test generative language models on a vast range of different evaluation tasks.

The following sections show you how to create an **LMEvalJob** custom resource (CR) which allows you to activate an evaluation job and generate an analysis of your model's ability.

2.1. SETTING UP LM-EVAL

LM-Eval is a service designed for evaluating large language models that has been integrated into the TrustyAI Operator.

The service is built on top of two open-source projects:

- LM Evaluation Harness, developed by EleutherAI, that provides a comprehensive framework for evaluating language models
- Unitxt, a tool that enhances the evaluation process with additional functionalities

The following information explains how to create an **LMEvalJob** custom resource (CR) to initiate an evaluation job and get the results.

Global settings for LM-Eval

Configurable global settings for LM-Eval services are stored in the TrustyAI operator global **ConfigMap**, named **trustyai-service-operator-config**. The global settings are located in the same namespace as the operator.

You can configure the following properties for LM-Eval:

Table 2.1. LM-Eval properties

Property	Default	Description
lmes-detect-device	true/false	Detect if there are GPUs available and assign a value for the --device argument for LM Evaluation Harness. If GPUs are available, the value is cuda . If there are no GPUs available, the value is cpu .
lmes-pod-image	quay.io/trustyai/talmes-job:latest	The image for the LM-Eval job. The image contains the Python packages for LM Evaluation Harness and Unitxt.

Property	Default	Description
lmes-driver-image	quay.io/trustyai/talmes-driver:latest	The image for the LM-Eval driver. For detailed information about the driver, see the cmd/lmes_driver directory.
lmes-image-pull-policy	Always	The image-pulling policy when running the evaluation job.
lmes-default-batch-size	8	The default batch size when invoking the model inference API. Default batch size is only available for local models.
lmes-max-batch-size	24	The maximum batch size that users can specify in an evaluation job.
lmes-pod-checking-interval	10s	The interval to check the job pod for an evaluation job.

After updating the settings in the **ConfigMap**, restart the operator to apply the new values.

2.2. ENABLING EXTERNAL RESOURCE ACCESS FOR LMEVAL JOBS

LMEval jobs do not allow internet access or remote code execution by default. When configuring an **LMEvalJob**, it may require access to external resources, for example task datasets and model tokenizers, usually hosted on [Hugging Face](#). If you trust the source and have reviewed the content of these artifacts, an **LMEvalJob** can be configured to automatically download them.

Follow the steps below to enable online access and remote code execution for LMEval jobs. Choose to update these settings by using either the CLI or in the console. Enable one or both settings according to your needs.

2.2.1. Enabling online access and remote code execution for LMEval Jobs using the CLI

You can enable online access using the CLI for LMEval jobs by setting the **allowOnline** specification to **true** in the **LMEvalJob** custom resource (CR). You can also enable remote code execution by setting the **allowCodeExecution** specification to **true**. Both modes can be used at the same time.



IMPORTANT

Enabling online access or code execution involves a security risk. Only use these configurations if you trust the source(s).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have downloaded and installed the OpenShift AI command-line interface (CLI). See [Installing the OpenShift CLI](#).

Procedure

1. Get the current **DataScienceCluster** resource, which is located in the **redhat-ods-operator** namespace:

```
$ oc get datasciencecluster -n redhat-ods-operator
```

Example output

```
NAME          AGE
default-dsc   10d
```

2. Enable online access and code execution for the cluster in the **DataScienceCluster** resource with the **permitOnline** and **permitCodeExecution** specifications. For example, create a file named **allow-online-code-exec-dsc.yaml** with the following contents:

Example **allow-online-code-exec-dsc.yaml** resource enabling online access and remote code execution

```
apiVersion: datasciencecluster.opendatahub.io/v1
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  # ...
  components:
    trustyai:
      managementState: Managed
    eval:
      lmeval:
        permitOnline: allow
        permitCodeExecution: allow
  # ...
```

The **permitCodeExecution** and **permitOnline** settings are disabled by default with a value of **deny**. You must explicitly enable these settings in the **DataScienceCluster** resource for the **LMEvalJob** instance to enable internet access or permission to run any externally downloaded code.

3. Apply the updated **DataScienceCluster**:

```
$ oc apply -f allow-online-code-exec-dsc.yaml -n redhat-ods-operator
```

- a. Optional: Run the following command to check that the **DataScienceCluster** is in a healthy state:

```
$ oc get datasciencecluster default-dsc
```

Example output

```
NAME      READY  REASON
default-dsc  True
```

- For new LMEval jobs, define the job in a YAML file as shown in the following example. This configuration requests both internet access, with **allowOnline: true**, and permission for remote code execution with, **allowCodeExecution: true**:

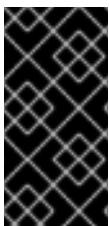
Example lmevaljob-with-online-code-exec.yaml

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: lmevaljob-with-online-code-exec
  namespace: <your_namespace>
spec:
  # ...
  allowOnline: true
  allowCodeExecution: true
  # ...
```

The **allowOnline** and **allowCodeExecution** settings are disabled by default with a value of **false** in the **LMEvalJob** CR.

- Deploy the LMEval Job:

```
$ oc apply -f lmevaljob-with-online-code-exec.yaml -n <your_namespace>
```



IMPORTANT

If you upgrade to version 2.25, some TrustyAI **LMEvalJob** CR configuration values might be overwritten. The new deployment prioritizes the value on the 2.25 version **DataScienceCluster**. Existing LMEval jobs are unaffected. Verify that all **DataScienceCluster** values are explicitly defined and validated during installation.

Verification

- Run the following command to verify that the **DataScienceCluster** has the updated fields:

```
$ oc get datasciencecluster default-dsc -n redhat-ods-operator -o "jsonpath={.data}"
```

- Run the following command to verify that the **trustyai-dsc-config** ConfigMap has the same flag values set in the **DataScienceCluster**.

```
$ oc get configmaps trustyai-dsc-config -n redhat-ods-applications -o "jsonpath={.spec.components.trustyai.eval.lmeval}"
```

Example output

```
■
```

```
{"eval.lmeval.permitCodeExecution":"true","eval.lmeval.permitOnline":"true"}
```

2.2.2. Updating LMEval job configuration using the web console

Follow these steps to enable online access (**allowOnline**) and remote code execution (**allowCodeExecution**) modes through the OpenShift AI web console for LMEval jobs.



IMPORTANT

Enabling online access or code execution involves a security risk. Only use these configurations if you trust the source(s).

Prerequisites

- You have cluster administrator privileges for your Red Hat OpenShift AI cluster.

Procedure

- In the OpenShift console, click **Operators** → **Installed Operators**.
- Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
- Click the **Data Science Cluster** tab.
- Click the default instance name (for example, **default-dsc**) to open the instance details page.
- Click the **YAML** tab to show the instance specifications.
- In the **spec:components:trustyai:eval:lmeval** section, set the **permitCodeExecution** and **permitOnline** fields to a value of **allow**:

```
spec:
  components:
    trustyai:
      managementState: Managed
    eval:
      lmeval:
        permitOnline: allow
        permitCodeExecution: allow
```

- Click **Save**.
- From the **Project** drop-down list, select the project that contains the LMEval job you are working with.
- From the **Resources** drop-down list, select the **LMEvalJob** instance that you are working with.
- Click **Actions** → **Edit YAML**
- Ensure that the **allowOnline** and **allowCodeExecution** are set to **true** to enable online access and code execution for this job when writing your **LMEvalJob** custom resource:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
```

```

kind: LMEvalJob
metadata:
  name: example-lmeval
spec:
  allowOnline: true
  allowCodeExecution: true

```

12. Click **Save**.

Table 2.2. Configuration keys for LMEvalJob custom resource

Field	Default	Description
spec.allowOnline	false	Enables this job to access the internet (e.g., to download datasets or tokenizers).
spec.allowCodeExecution	false	Allows this job to run code included with downloaded resources.

2.3. LM-EVAL EVALUATION JOB

LM-Eval service defines a new Custom Resource Definition (CRD) called **LMEvalJob**. An **LMEvalJob** object represents an evaluation job. **LMEvalJob** objects are monitored by the TrustyAI Kubernetes operator.

To run an evaluation job, create an **LMEvalJob** object with the following information: **model**, **model arguments**, **task**, and **secret**.



NOTE

For a list of TrustyAI-supported tasks, see [LMEval task support](#).

After the **LMEvalJob** is created, the LM-Eval service runs the evaluation job. The status and results of the **LMEvalJob** object update when the information is available.



NOTE

Other TrustyAI features (such as bias and drift metrics) cannot be used with non-tabular models (including LLMs). Deploying the **TrustyAIService** custom resource (CR) in a namespace that contains non-tabular models (such as the namespace where an evaluation job is being executed) can cause errors within the TrustyAI service.

Sample LMEvalJob object

The sample **LMEvalJob** object contains the following features:

- The **google/flan-t5-base** model from Hugging Face.
- The dataset from the **wnli** card, a subset of the GLUE (General Language Understanding Evaluation) benchmark evaluation framework from Hugging Face. For more information about the **wnli** Unitxt card, see the [Unitxt website](#).

- The following default parameters for the **multi_class.relation** Unitxt task: **f1_micro**, **f1_macro**, and **accuracy**. This template can be found on the Unitxt website: click **Catalog**, then click **Tasks** and select **Classification** from the menu.

The following is an example of an **LMEvalJob** object:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
    - name: pretrained
      value: google/flan-t5-base
  taskList:
    taskRecipes:
      - card:
          name: "cards.wnli"
          template: "templates.classification.multi_class.relation.default"
  logSamples: true
```

After you apply the sample **LMEvalJob**, check its state by using the following command:

```
oc get lmevaljob evaljob-sample
```

Output similar to the following appears: NAME: **evaljob-sample** STATE: **Running**

Evaluation results are available when the state of the object changes to **Complete**. Both the model and dataset in this example are small. The evaluation job should finish within 10 minutes on a CPU-only node.

Use the following command to get the results:

```
oc get lmevaljobs.trustyai.opendatahub.io evaljob-sample \
-o template --template={{.status.results}} | jq '.results'
```

The command returns results similar to the following example:

```
{
  "tr_0": {
    "alias": "tr_0",
    "f1_micro,none": 0.5633802816901409,
    "f1_micro_stderr,none": "N/A",
    "accuracy,none": 0.5633802816901409,
    "accuracy_stderr,none": "N/A",
    "f1_macro,none": 0.36036036036036034,
    "f1_macro_stderr,none": "N/A"
  }
}
```

Notes on the results

- The **f1_micro**, **f1_macro**, and **accuracy** scores are 0.56, 0.36, and 0.56.

- The full results are stored in the **.status.results** of the **LMEvalJob** object as a JSON document.
- The command above only retrieves the results field of the JSON document.



NOTE

The provided **LMEvalJob** uses a dataset from the **wnli** card, which is in Parquet format and not supported on **s390x**. To run on **s390x**, choose a task that uses a non-Parquet dataset.

2.4. LM-EVAL EVALUATION JOB PROPERTIES

The **LMEvalJob** object contains the following features:

- The **google/flan-t5-base** model.
- The dataset from the **wnli** card, from the GLUE (General Language Understanding Evaluation) benchmark evaluation framework.
- The **multi_class.relation** Unitxt task default parameters.

The following table lists each property in the **LMEvalJob** and its usage:

Table 2.3. LM-EvalJob properties

Parameter	Description
model	Specifies which model type or provider is evaluated. This field directly maps to the --model argument of the lm-evaluation-harness. The model types and providers that you can use include: • hf: HuggingFace models • openai-completions: OpenAI Completions API models • openai-chat-completions: OpenAI Chat Completions API models • local-completions and local-chat-completions: OpenAI API-compatible servers • textsynth: TextSynth APIs

Parameter	Description
modelArgs	A list of paired name and value arguments for the model type. Arguments vary by model provider. You can find further details in the models section of the LM Evaluation Harness library on GitHub. Below are examples for some providers: ● hf: The model designation for the HuggingFace provider ● local-completions: An OpenAI API-compatible server ● local-chat-completions: An OpenAI API-compatible server ● openai-completions: OpenAI Completions API models ● openai-chat-completions: ChatCompletions API models ● textsynth: TextSynth APIs
taskList.taskNames	Specifies a list of tasks supported by lm-evaluation-harness .

Parameter	Description
taskList.taskRecipes	Specifies the task using the Unitxt recipe format: ● card: Use the name to specify a Unitxt card or ref to refer to a custom card: ○ name: Specifies a Unitxt card from the catalog section of the Unitxt. Use the card ID as the value. For example, the ID of the Wnli card is cards.wnli. ○ ref: Specifies the reference name of a custom card as defined in the custom section. If the dataset used by the custom card requires an API key from an environment variable or a persistent volume, configure the necessary resources in the pod field. ● template: Specifies a Unitxt template from the Unitxt catalog. Use name to specify a Unitxt catalog template or ref to refer to a custom template: ○ name: Specifies a Unitxt template from the catalog of cards on the Unitxt website. Use the template's ID as the value. ○ ref: Specifies the reference name of a custom template as defined in the custom section. ● systemPrompt: Use name to specify a Unitxt catalog system prompt or ref to refer to a custom prompt: ○ name: Specifies a Unitxt system prompt from the catalog on the Unitxt website. Use the system prompt's ID as the value. ○ ref: Specifies the reference name of a custom system prompt as defined in the custom section. ● task (optional): Specifies a Unitxt task from the Unitxt catalog. Use the task ID as the value. A Unitxt card has a predefined task. Only specify a value for this if you want to run a different task. ● metrics (optional): Specifies a Unitxt task from the Unitxt catalog. Use the metric ID as the value. A Unitxt task has a set of pre-defined metrics. Only specify a set of metrics if you need different metrics. ● format (optional): Specifies a Unitxt format from the Unitxt catalog. Use the format ID as the value. ● loaderLimit (optional): Specifies the maximum number of instances per stream to be returned from the loader. You can use this parameter to reduce loading time in large datasets. ● numDemos (optional): Number of few-shot to be used. ● demosPoolSize (optional): Size of the few-shot pool.
numFewShot	Sets the number of few-shot examples to place in context. If you are using a task from Unitxt, do not use this field. Use numDemos under the taskRecipes instead.

Parameter	Description
limit	Set a limit to run the tasks instead of running the entire dataset. Accepts either an integer or a float between 0.0 and 1.0 .
genArgs	Maps to the --gen_kwargs parameter for the lm-evaluation-harness . For more information, see the LM Evaluation Harness documentation on GitHub.
logSamples	If this flag is passed, then the model outputs and the text fed into the model are saved at per-prompt level.
batchSize	Specifies the batch size for the evaluation in integer format. The auto:N batch size is not used for API models, but numeric batch sizes are used for APIs.
pod	Specifies extra information for the lm-eval job pod: • container: Specifies additional container settings for the lm-eval container. ◦ env: Specifies environment variables. This parameter uses the EnvVar data structure of Kubernetes. ◦ volumeMounts: Mounts the volumes into the lm-eval container. ◦ resources: Specifies the resources for the lm-eval container. • volumes: Specifies the volume information for the lm-eval and other containers. This parameter uses the Volume data structure of Kubernetes. • sideCars: A list of containers that run along with the lm-eval container. This parameter uses the Container data structure of Kubernetes.
outputs	This parameter defines a custom output location to store the the evaluation results. Only Persistent Volume Claims (PVC) are supported.
outputs.pvcManaged	Creates an operator-managed PVC to store the job results. The PVC is named <job-name>-pvc and is owned by the LMEvalJob. After the job finishes, the PVC is still available, but it is deleted with the LMEvalJob. Supports the following fields: • size: The PVC size, compatible with standard PVC syntax (for example, 5Gi).
outputs.pvcName	Binds an existing PVC to a job by specifying its name. The PVC must be created separately and must already exist when creating the job.

Parameter	Description
allowOnline	If this parameter is set to true , the LMEval job downloads artifacts as needed (for example, models, datasets or tokenizers). If set to false , artifacts are not downloaded and are pulled from local storage instead. This setting is disabled by default. If you want to enable allowOnline mode, you can deploy a new LMEvalJob CR with allowOnline set to true as long as the DataScienceCluster resource specification permitOnline is also set to true .
allowCodeExecution	If this parameter is set to true , the LMEval job runs the necessary code for preparing models or datasets. If set to false it does not run downloaded code. The default setting for this parameter is false . If you want to enable allowCodeExecution mode, you can deploy a new LMEvalJob CR with allowCodeExecution set to true as long as the DataScienceCluster resource specification permitCodeExecution is also set to true .
offline	Mount a PVC as the local storage for models and datasets.
systemInstruction	(Optional) Sets the system instruction for all prompts passed to the evaluated model.
chatTemplate	Applies the specified chat template to prompts. Contains two fields: * enabled : If set to true , a chat template is used. If set to false , no template is used. * name : Uses the template name, if provided. If no name argument is provided, uses the default template for the model.

2.4.1. Properties for setting up custom Unitxt cards, templates, or system prompts

You can choose to set up custom Unitxt cards, templates, or system prompts. Use the parameters set out in the Custom Unitxt parameters table in addition to the preceding table parameters to set customized Unitxt items:

Table 2.4. Custom Unitxt parameters

Parameter	Description
-----------	-------------

Parameter	Description
taskList.custom	Defines one or more custom resources that is referenced in a task recipe. The following custom cards, templates, and system prompts are supported: ● cards: Defines custom cards to use, each with name and value field: ○ name: The name of this custom card that is referenced in the card.ref field of a task recipe. ○ value: A JSON string for a custom Unitxt card that contains the custom dataset. To compose a custom card, store it as a JSON file, and use the JSON content as the value. If the dataset used by the custom card needs an API key from an environment variable or a persistent volume, set up corresponding resources under the pod field in the LMEvalJob properties table. ● templates: Define custom templates to use, each with name and value field: ○ name: The name of this custom template that is referenced in the template.ref field of a task recipe. ○ value: A JSON string for a custom Unitxt template. Store value as a JSON file and use the JSON content as the value of this field. ● systemPrompts: Defines custom system prompts to use, each with a name and value field: ○ name: The name of this custom system prompt that is referenced in the systemPrompt.ref field of a task recipe. ○ value: A string for a custom Unitxt system prompt. You can see an overview of the different components that make up a prompt format, including the system prompt, on the Unitxt website.

2.5. PERFORMING MODEL EVALUATIONS IN THE DASHBOARD

LM-Eval is a Language Model Evaluation as a Service (LM-Eval-aaS) feature integrated into the TrustyAI Operator. It offers a unified framework for testing generative language models across a wide variety of evaluation tasks. You can use LM-Eval through the Red Hat OpenShift AI dashboard or the OpenShift CLI (**oc**). These instructions are for using the dashboard.



IMPORTANT

Model evaluation through the dashboard is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have logged in to Red Hat OpenShift AI with administrator privileges.
- You have enabled the TrustyAI component, as described in [Enabling the TrustyAI component](#).
- You have created a project in OpenShift AI.
- You have deployed an LLM model in your project.



NOTE

By default, the **Develop & train → Evaluations** page is hidden from the dashboard navigation menu. To show the **Develop & train → Evaluations** page in the dashboard, go to the **OdhDashboardConfig** custom resource (CR) in Red Hat OpenShift AI and set the **disableLMEval** value to false. For more information about enabling dashboard configuration options, see [Dashboard configuration options](#).

Procedure

1. In the dashboard, click **Develop & train → Evaluations**. The Evaluations page opens. It contains:
 - a. A **Start evaluation run** button. If you have not run any previous evaluations, only this button is displayed.
 - b. A list of evaluations you have previously run, if any exist.
 - c. A **Project** dropdown option you can click to show the evaluations relating to one project instead of all projects.
 - d. A filter to sort your evaluations by model or evaluation name.

The following table outlines the elements and functions of the evaluations list:

Table 2.5. Evaluations list components

Property	Function
Evaluation	The name of the evaluation.
Model	The model that was used in the evaluation.
Evaluated	The date and time when the evaluation was created.

Property	Function
Status	The status of your evaluation: running, completed, or failed.
More options icon	Click this icon to access the options to delete the evaluation, or download the evaluation log in JSON format.

2. From the **Project** dropdown menu, select the namespace of the project where you want to evaluate the model.
3. Click the **Start evaluation run** button. The Model evaluation form is displayed.
4. Fill in the details of the form. The model argument summary is displayed after you complete the form details:
 - a. **Model name:** Select a model from all the deployed LLMs in your project.
 - b. **Evaluation name:** Give your evaluation a unique name.
 - c. **Tasks:** Choose one or more evaluation tasks against which to measure your LLM. The 100 most common evaluation tasks are supported.
 - d. **Model type:** Choose the type of model based on the type of prompt-formatting you use:
 - i. **Local-completion:** You assemble the entire prompt chain yourself. Use this when you want to evaluate models that take a plain text prompt and return a continuation.
 - ii. **Local-chat-completion:** The framework injects roles or templates automatically. Use this for models that simulate a conversation by taking a list of chat messages with roles like **user** and **assistant** and reply appropriately.
 - e. **Security settings:**
 - i. **Available online:** Choose **enable** to allow your model to access the internet to download datasets.
 - ii. **Trust remote code:** Choose **enable** to allow your model to trust code from outside of the project namespace.



NOTE

The **Security settings** section is grayed out if the security option in global settings is set to **active**.

5. Observe that a model argument summary is displayed as soon as you fill in the form details.
6. Complete the tokenizer settings:
 - a. **Tokenized requests:** If set to **true**, the evaluation requests are broken down into tokens. If set to **false**, the evaluation dataset remains as raw text.
 - b. **Tokenizer:** Type the model's tokenizer URL that is required for the evaluations.

- Click **Evaluate**. The screen returns to the model evaluation page of your project and your job is displayed in the evaluations list.



NOTE

- It can take time for your evaluation to complete, depending on factors including hardware support, model size, and the type of evaluation task(s). The status column reports the current status of the evaluation: *completed*, *running*, or *failed*.
- If your evaluation fails, the evaluation pod logs in your cluster provide more information.

2.6. LM-EVAL SCENARIOS

The following procedures outline example scenarios that can be useful for an LM-Eval setup.

2.6.1. Accessing Hugging Face models with an environment variable token

If the **LMEvalJob** needs to access a model on HuggingFace with the access token, you can set up the **HF_TOKEN** as one of the environment variables for the **lm-eval** container.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

Procedure

- To start an evaluation job for a **huggingface** model, apply the following YAML file to your project through the CLI:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
    - name: pretrained
      value: huggingfacespace/model
  taskList:
    taskNames:
      - unfair_tos/
  logSamples: true
  pod:
    container:
      env:
        - name: HF_TOKEN
          value: "My HuggingFace token"
```

For example:

```
$ oc apply -f <yaml_file> -n <project_name>
```

2. (Optional) You can also create a secret to store the token, then refer the key from the **secretKeyRef** object using the following reference syntax:

```
env:
  - name: HF_TOKEN
    valueFrom:
      secretKeyRef:
        name: my-secret
        key: hf-token
```

2.6.2. Using a custom Unitxt card

You can run evaluations using custom Unitxt cards. To do this, include the custom Unitxt card in JSON format within the **LMEvalJob** YAML.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

Procedure

1. Pass a custom Unitxt Card in JSON format:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
    - name: pretrained
      value: google/flan-t5-base
  taskList:
    taskRecipes:
      - template: "templates.classification.multi_class.relation.default"
    card:
      custom: |
        {
          "__type__": "task_card",
          "loader": {
            "__type__": "load_hf",
            "path": "glue",
            "name": "wnli"
          },
          "preprocess_steps": [
            {
              "__type__": "split_random_mix",
              "mix": {
                "train": "train[95%]",
```



```

        "validation": "train[5%]",
        "test": "validation"
    }
},
{
    "__type__": "rename",
    "field": "sentence1",
    "to_field": "text_a"
},
{
    "__type__": "rename",
    "field": "sentence2",
    "to_field": "text_b"
},
{
    "__type__": "map_instance_values",
    "mappers": {
        "label": {
            "0": "entailment",
            "1": "not entailment"
        }
    }
},
{
    "__type__": "set",
    "fields": {
        "classes": [
            "entailment",
            "not entailment"
        ]
    }
},
{
    "__type__": "set",
    "fields": {
        "type_of_relation": "entailment"
    }
},
{
    "__type__": "set",
    "fields": {
        "text_a_type": "premise"
    }
},
{
    "__type__": "set",
    "fields": {
        "text_b_type": "hypothesis"
    }
}
],
"task": "tasks.classification.multi_class.relation",
"templates": "templates.classification.multi_class.relation.all"
}
logSamples: true

```

2. Inside the custom card specify the Hugging Face dataset loader:

```
"loader": {
  "__type__": "load_hf",
  "path": "glue",
  "name": "wnli"
},
```

3. (Optional) You can use other Unitxt loaders (found on the Unitxt website) that contain the **volumes** and **volumeMounts** parameters to mount the dataset from persistent volumes. For example, if you use the **LoadCSV** Unitxt command, mount the files to the container and make the dataset accessible for the evaluation process.



NOTE

The provided scenario example does not work on **s390x**, as it uses a Parquet-type dataset, which is not supported on this architecture. To run the scenario on **s390x**, use a task with a non-Parquet dataset.

2.6.3. Using PVCs as storage

To use a PVC as storage for the **LMEvalJob** results, you can use either managed PVCs or existing PVCs. Managed PVCs are managed by the TrustyAI operator. Existing PVCs are created by the end-user before the **LMEvalJob** is created.



NOTE

If both managed and existing PVCs are referenced in outputs, the TrustyAI operator defaults to the managed PVC.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

2.6.3.1. Managed PVCs

To create a managed PVC, specify its size. The managed PVC is named **<job-name>-pvc** and is available after the job finishes. When the **LMEvalJob** is deleted, the managed PVC is also deleted.

Procedure

- Enter the following code:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  # other fields omitted ...
```

```

outputs:
  pvcManaged:
    size: 5Gi

```

Notes on the code

- **outputs** is the section for specifying custom storage locations
- **pvcManaged** will create an operator-managed PVC
- **size** (compatible with standard PVC syntax) is the only supported value

2.6.3.2. Existing PVCs

To use an existing PVC, pass its name as a reference. The PVC must exist when you create the **LMEvalJob**. The PVC is not managed by the TrustyAI operator, so it is available after deleting the **LMEvalJob**.

Procedure

1. Create a PVC. An example is the following:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: "my-pvc"
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

```

2. Reference the new PVC from the **LMEvalJob**.

```

apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  # other fields omitted ...
  outputs:
    pvcName: "my-pvc"

```

2.6.4. Using a KServe Inference Service

To run an evaluation job on an **InferenceService** which is already deployed and running in your namespace, define your **LMEvalJob** CR, then apply this CR into the same namespace as your model.

NOTE

The following example only works with Hugging Face or vLLM-based model-serving runtimes.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You have a namespace that contains an InferenceService with a vLLM model. This example assumes that a vLLM model is already deployed in your cluster.
- Your cluster has Domain Name System (DNS) configured.

Procedure

1. Define your **LMEvalJob** CR:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob
spec:
  model: local-completions
  taskList:
    taskNames:
      - mmlu
  logSamples: true
  batchSize: 1
  modelArgs:
    - name: model
      value: granite
    - name: base_url
      value: $ROUTE_TO_MODEL/v1/completions
    - name: num_concurrent
      value: "1"
    - name: max_retries
      value: "3"
    - name: tokenized_requests
      value: false
    - name: tokenizer
      value: huggingfacespace/model
  env:
    - name: OPENAI_TOKEN
      valueFrom:
        secretKeyRef:
          name: <secret-name>
          key: token
```

2. Apply this CR into the same namespace as your model.

Verification

A pod spins up in your model namespace called **evaljob**. In the pod terminal, you can see the output via **tail -f output/stderr.log**.

Notes on the code

- **base_url** should be set to the route/service URL of your model. Make sure to include the **/v1/completions** endpoint in the URL.
- **env.valueFrom.secretKeyRef.name** should point to a secret that contains a token that can authenticate to your model. **secretRef.name** should be the secret's name in the namespace, while **secretRef.key** should point at the token's key within the secret.
- **secretKeyRef.name** can equal the output of:

```
oc get secrets -o custom-columns=SECRET:.metadata.name --no-headers | grep user-one-token
```

- **secretKeyRef.key** is set to **token**

2.6.5. Setting up LM-Eval S3 Support

Learn how to set up S3 support for your LM-Eval service.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You have a namespace that contains an S3-compatible storage service and bucket.
- You have created an **LMEvalJob** that references the S3 bucket containing your model and dataset.
- You have an S3 bucket that contains the model files and the dataset(s) to be evaluated.

Procedure

1. Create a Kubernetes Secret containing your S3 connection details:

```
apiVersion: v1
kind: Secret
metadata:
  name: "s3-secret"
  namespace: test
  labels:
    opendatahub.io/dashboard: "true"
    opendatahub.io/managed: "true"
  annotations:
    opendatahub.io/connection-type: s3
    openshift.io/display-name: "S3 Data Connection - LMEval"
data:
  AWS_ACCESS_KEY_ID: BASE64_ENCODED_ACCESS_KEY # Replace with your key
  AWS_SECRET_ACCESS_KEY: BASE64_ENCODED_SECRET_KEY # Replace with
your key
  AWS_S3_BUCKET: BASE64_ENCODED_BUCKET_NAME # Replace with your bucket
name
  AWS_S3_ENDPOINT: BASE64_ENCODED_ENDPOINT # Replace with your endpoint
```

URL (for example, <https://s3.amazonaws.com>)
 AWS_DEFAULT_REGION: BASE64_ENCODED_REGION # Replace with your region
 type: Opaque



NOTE

All values must be **base64** encoded. For example: **echo -n "my-bucket" | base64**

2. Deploy the **LMEvalJob** CR that references the S3 bucket containing your model and dataset:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  allowOnline: false
  model: hf # Model type (HuggingFace in this example)
  modelArgs:
    - name: pretrained
      value: /opt/app-root/src/hf_home/flan # Path where model is mounted in container
  taskList:
    taskNames:
      - arc_easy # The evaluation task to run
  logSamples: true
  offline:
    storage:
      s3:
        accessKeyId:
          name: s3-secret
          key: AWS_ACCESS_KEY_ID
        secretAccessKey:
          name: s3-secret
          key: AWS_SECRET_ACCESS_KEY
        bucket:
          name: s3-secret
          key: AWS_S3_BUCKET
        endpoint:
          name: s3-secret
          key: AWS_S3_ENDPOINT
        region:
          name: s3-secret
          key: AWS_DEFAULT_REGION
        path: "" # Optional subfolder within bucket
        verifySSL: false
```



IMPORTANT

The ``LMEvalJob`` will copy all the files from the specified bucket/path. If your bucket contains many files and you only want to use a subset, set the ``path`` field to the specific sub-folder containing the files that you require. For example use ``path: "my-models/"``.

3. Set up a secure connection using SSL.

- a. Create a ConfigMap object with your CA certificate:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: s3-ca-cert
  namespace: test
  annotations:
    service.beta.openshift.io/inject-cabundle: "true" # For injection
data: {} # OpenShift will inject the service CA bundle
# Or add your custom CA:
# data:
#   ca.crt: |-
#     -----BEGIN CERTIFICATE-----
#     ...your CA certificate content...
#     -----END CERTIFICATE-----
```

- b. Update the **LMEvalJob** to use SSL verification:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  # ... same as above ...
  offline:
    storage:
      s3:
        # ... same as above ...
        verifySSL: true # Enable SSL verification
        caBundle:
          name: s3-ca-cert # ConfigMap name containing your CA
          key: service-ca.crt # Key in ConfigMap containing the certificate
```

Verification

1. After deploying the **LMEvalJob**, open the **kubectl** command-line and enter this command to check its status: **kubectl logs -n test job/evaljob-sample -n test**
2. View the logs with the **kubectl** command **kubectl logs -n test job/<job-name>** to make sure it has functioned correctly.
3. The results are displayed in the logs after the evaluation is completed.

2.6.6. Using LLM-as-a-Judge metrics with LM-Eval

You can use a large language model (LLM) to assess the quality of outputs from another LLM, known as LLM-as-a-Judge (LLMaaJ).

You can use LLMaaJ to:

- Assess work with no clearly correct answer, such as creative writing.

- Judge quality characteristics such as helpfulness, safety, and depth.
- Augment traditional quantitative measures that are used to evaluate a model's performance (for example, **ROUGE** metrics).
- Test specific quality aspects of your model output.

Follow the custom quality assessment example below to learn more about using your own metrics criteria with LM-Eval to evaluate model responses.

This example uses [Unitxt](#) to define custom metrics and to see how the model ([flan-t5-small](#)) answers questions from MT-Bench, a standard benchmark. Custom evaluation criteria and instructions from the [Mistral-7B](#) model are used to rate the answers from 1-10, based on helpfulness, accuracy, and detail.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You are familiar with how to use Unitxt.
- You have set the following parameters:

Table 2.6. Parameters

Parameter	Description
Custom template	Tells the judge to assign a score between 1 and 10 in a standardized format, based on specific criteria.
processors.extract_mt_bench_rating_judgment	Pulls the numerical rating from the judge's response.
formats.models.mistral.instruction	Formats the prompts for the Mistral model.
Custom LLM-as-judge metric	Uses Mistral-7B with your custom instructions.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```


2. Apply the following manifest by using the **oc apply -f** command. The YAML content defines a custom evaluation job (**LMEvalJob**), the namespace, and the location of the model you want to evaluate. The YAML contains the following instructions:
 - a. Which model to evaluate.
 - b. What data to use.
 - c. How to format inputs and outputs.
 - d. Which judge model to use.
 - e. How to extract and log results.



NOTE

You can also put the YAML manifest into a file using a text editor and then apply it by using the **oc apply -f file.yaml** command.

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: custom-eval
  namespace: test
spec:
  allowOnline: true
  allowCodeExecution: true
  model: hf
  modelArgs:
    - name: pretrained
      value: google/flan-t5-small
  taskList:
  taskRecipes:
    - card:
        custom: |
          {
            "__type__": "task_card",
            "loader": {
              "__type__": "load_hf",
              "path": "OfirArviv/mt_bench_single_score_gpt4_judgement",
              "split": "train"
            },
            "preprocess_steps": [
              {
                "__type__": "rename_splits",
                "mapper": {
                  "train": "test"
                }
              },
              {
                "__type__": "filter_by_condition",
                "values": {
                  "turn": 1
                },
                "condition": "eq"
              }
            ]
          }
```

```

    },
    {
      "__type__": "filter_by_condition",
      "values": {
        "reference": "[]"
      },
      "condition": "eq"
    },
    {
      "__type__": "rename",
      "field_to_field": {
        "model_input": "question",
        "score": "rating",
        "category": "group",
        "model_output": "answer"
      }
    },
    {
      "__type__": "literal_eval",
      "field": "question"
    },
    {
      "__type__": "copy",
      "field": "question/0",
      "to_field": "question"
    },
    {
      "__type__": "literal_eval",
      "field": "answer"
    },
    {
      "__type__": "copy",
      "field": "answer/0",
      "to_field": "answer"
    }
  ],
  "task": "tasks.response_assessment.rating.single_turn",
  "templates": [
    "templates.response_assessment.rating.mt_bench_single_turn"
  ]
}

```

template:

ref: response_assessment.rating.mt_bench_single_turn

format: formats.models.mistral.instruction

metrics:

- ref: llama_j_metric

custom:

templates:

- name: response_assessment.rating.mt_bench_single_turn

value: |

```

{
  "__type__": "input_output_template",
  "instruction": "Please act as an impartial judge and evaluate the quality of the response

```

provided by an AI assistant to the user question displayed below. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level of detail of the response. Begin your evaluation by providing a short explanation. Be as objective as possible. After

```

providing your explanation, you must rate the response on a scale of 1 to 10 by strictly following this
format: \"[[rating]]\", for example: \"Rating: [[5]]\".\n\n",
    "input_format": "[Question]\n{question}\n\n[The Start of Assistant's Answer]\n{answer}\n[The
End of Assistant's Answer]",
    "output_format": "[[[rating]]]",
    "postprocessors": [
        "processors.extract_mt_bench_rating_judgment"
    ]
}
tasks:
- name: response_assessment.rating.single_turn
  value: |
    {
      "__type__": "task",
      "input_fields": {
        "question": "str",
        "answer": "str"
      },
      "outputs": {
        "rating": "float"
      },
      "metrics": [
        "metrics.spearman"
      ]
    }
metrics:
- name: llmaaj_metric
  value: |
    {
      "__type__": "llm_as_judge",
      "inference_model": {
        "__type__": "hf_pipeline_based_inference_engine",
        "model_name": "mistralai/Mistral-7B-Instruct-v0.2",
        "max_new_tokens": 256,
        "use_fp16": true
      },
      "template": "templates.response_assessment.rating.mt_bench_single_turn",
      "task": "rating.single_turn",
      "format": "formats.models.mistral.instruction",
      "main_score": "mistral_7b_instruct_v0_2_huggingface_template_mt_bench_single_turn"
    }
logSamples: true
pod:
  container:
    env:
      - name: HF_TOKEN
        valueFrom:
          secretKeyRef:
            name: hf-token-secret
            key: token
  resources:
    limits:
      cpu: '2'
      memory: 16Gi

```

Verification

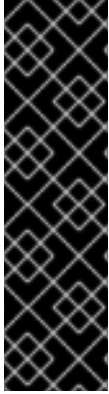
A processor extracts the numeric rating from the judge's natural language response. The final result is available as part of the LMEval Job Custom Resource (CR).



NOTE

The provided scenario example does not work for **s390x**. The scenario works with non-Parquet type dataset task for **s390x**.

CHAPTER 3. EVALUATING RAG SYSTEMS WITH RAGAS



IMPORTANT

Retrieval-Augmented Generation Assessment (Ragas) is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

As an AI engineer, you can use Retrieval-Augmented Generation Assessment ([Ragas](#)) to measure and improve the quality of your RAG systems in OpenShift AI. Ragas provides objective metrics that assess retrieval quality, answer relevance, and factual consistency, enabling you to identify issues, optimize configurations, and establish automated quality gates in your development workflows.

Ragas is integrated with OpenShift AI through the Llama Stack evaluation API and supports two deployment modes: an inline provider for development and testing, and a remote provider for production-scale evaluations using OpenShift AI pipelines.

3.1. ABOUT RAGAS EVALUATION

Ragas addresses the unique challenges of evaluating RAG systems by providing metrics that assess both the retrieval and generation components of your application. Unlike traditional language model evaluation that focuses solely on output quality, Ragas evaluates how well your system retrieves relevant context and generates responses grounded in that context.

3.1.1. Key Ragas metrics

Ragas provides [multiple metrics](#) for evaluating RAG systems. Here are some of the metrics:

Faithfulness

Measures the generated answer to determine whether it is consistent with the retrieved context. A high faithfulness score indicates that the answer is well-grounded in the source documents, reducing the risk of hallucinations. This is critical for enterprise and regulated environments where accuracy and trustworthiness are paramount.

Answer Relevancy

Evaluates whether the generated answer is consistent with the input question. This metric ensures that your RAG system provides pertinent responses rather than generic or off-topic information.

Context Precision

Measures the precision of the retrieval component by evaluating whether the retrieved context chunks contain information relevant to answering the question. High precision indicates that your retrieval system is returning focused, relevant documents rather than irrelevant noise.

Context Recall

Measures the recall of the retrieval component by evaluating whether all necessary information for answering the question is present in the retrieved contexts. High recall ensures that your retrieval system is not missing important information.

Answer Correctness

Compares the generated answer with a ground truth reference answer to measure accuracy. This metric is useful when you have labeled evaluation datasets with known correct answers.

Answer Similarity

Measures the semantic similarity between the generated answer and a reference answer, providing a more nuanced assessment than exact string matching.

3.1.2. Use cases for Ragas in AI engineering workflows

Ragas enables AI engineers to accomplish the following tasks:

Automate quality checks

Create reproducible, objective evaluation jobs that can be automatically triggered after every code commit or model update. Automatic quality checks establish quality gates to prevent regressions and ensure that you deploy only high-quality RAG configurations to production.

Enable evaluation-driven development (EDD)

Use Ragas metrics to guide iterative optimization. For example, test different chunking strategies, embedding models, or retrieval algorithms against a defined benchmark. You can discover the optimal RAG configuration that maximizes performance metrics. For example, you can maximize faithfulness while minimizing computational cost.

Ensure factual consistency and trustworthiness

Measure the reliability of your RAG system by setting thresholds on metrics like faithfulness. Metrics thresholds ensure that responses are consistently grounded in source documents, which is critical for enterprise applications where hallucinations or factual errors are unacceptable.

Achieve production scalability

Leverage the remote provider pattern with OpenShift AI pipelines to execute evaluations as distributed jobs. The remote provider pattern allows you to run large-scale benchmarks across thousands of data points without blocking development or consuming excessive local resources.

Compare model and configuration variants

Run comparative evaluations across different models, retrieval strategies, or system configurations to make data-driven decisions about your RAG architecture. For example, compare the impact of different chunk sizes (512 vs 1024 tokens) or different embedding models on retrieval quality metrics.

3.1.3. Ragas provider deployment modes

OpenShift AI supports two deployment modes for Ragas evaluation:

Inline provider

The inline provider mode runs Ragas evaluation in the same process as the Llama Stack server. Use the inline provider for development and rapid prototyping. It offers the following advantages:

- Fast processing with in-memory operations
- Minimal configuration overhead
- Local development and testing
- Evaluation of small to medium-sized datasets

Remote provider

The remote provider mode runs Ragas evaluation as distributed jobs using OpenShift AI pipelines (powered by KubeFlow Pipelines). Use the remote provider for production environments. It offers the following capabilities:

- Running evaluations in parallel across thousands of data points
- Providing resource isolation and management
- Integrating with CI/CD pipelines for automated quality gates
- Storing results in S3-compatible object storage
- Tracking evaluation history and metrics over time
- Supporting large-scale batch evaluations without impacting the Llama Stack server

3.2. SETTING UP THE RAGAS INLINE PROVIDER FOR DEVELOPMENT

You can set up the Ragas inline provider to run evaluations directly within the Llama Stack server process. The inline provider is ideal for development environments, rapid prototyping, and lightweight evaluation workloads where simplicity and quick iteration are priorities.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed a Llama model with KServe. For more information, see [Deploying a Llama model with KServe](#).
- You have created a project.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Navigate to your project:

```
$ oc project <project_name>
```

3. Create a ConfigMap for the Ragas inline provider configuration. For example, create a **ragas-inline-config.yaml** file as follows:

Example ragas-inline-config.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: ragas-inline-config
  namespace: <project_name>
data:
  EMBEDDING_MODEL: "all-MiniLM-L6-v2"

```

- **EMBEDDING_MODEL**: Used by Ragas for semantic similarity calculations. The **all-MiniLM-L6-v2** model is a lightweight, efficient option suitable for most use cases.

4. Apply the ConfigMap:

```
$ oc apply -f ragas-inline-config.yaml
```

5. Create a Llama Stack distribution configuration file with the Ragas inline provider. For example, create a **llama-stack-ragas-inline.yaml** file as follows:

Example llama-stack-ragas-inline.yaml

```

apiVersion: llamastack.trustyai.opendatahub.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llama-stack-ragas-inline
  namespace: <project_name>
spec:
  replicas: 1
  server:
    containerSpec:
      env:
        # ...
        - name: VLLM_URL
          value: <model_url>
        - name: VLLM_API_TOKEN
          value: <model_api_token (if necessary)>
        - name: INFERENCE_MODEL
          value: <model_name>
        - name: MILVUS_DB_PATH
          value: ~/.llama/milvus.db
        - name: VLLM_TLS_VERIFY
          value: "false"
        - name: FMS_ORCHESTRATOR_URL
          value: http://localhost:123
        - name: EMBEDDING_MODEL
          value: granite-embedding-125m
        # ...

```

6. Deploy the Llama Stack distribution:

```
$ oc apply -f llama-stack-ragas-inline.yaml
```

7. Wait for the deployment to complete:


```
$ oc get pods -w
```

Wait until the **llama-stack-ragas-inline** pod status shows **Running**.

3.3. CONFIGURING THE RAGAS REMOTE PROVIDER FOR PRODUCTION

Next steps

You can configure the Ragas remote provider to run evaluations as distributed jobs using OpenShift AI pipelines. The remote provider enables production-scale evaluations by running Ragas in a separate KubeFlow Pipelines environment, providing resource isolation, improved scalability, and integration with CI/CD workflows.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift AI Operator.
- You have a **DataScienceCluster** custom resource in your environment; in the **spec.components** section the **llamastackoperator.managementState** is enabled with a value of **Managed**.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have configured a pipeline server in your project. For more information, see [Configuring a pipeline server](#).
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed a Large Language Model with KServe. For more information, see [Deploying a Llama model with KServe](#).
- You have configured S3-compatible object storage for storing evaluation results and you know your S3 credentials: AWS access key, AWS secret access key, and AWS default region. For more information, see [Adding a connection to your project](#).
- You have created a project.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Navigate to your project:

```
$ oc project <project_name>
```

3. Create a secret for storing S3 credentials:

```
$ oc create secret generic "<ragas_s3_credentials>" \
  --from-literal=AWS_ACCESS_KEY_ID=<your_access_key> \
  --from-literal=AWS_SECRET_ACCESS_KEY=<your_secret_key> \
  --from-literal=AWS_DEFAULT_REGION=<your_region>
```

IMPORTANT

Replace the placeholder values with your actual S3 credentials. These AWS credentials are required in two locations:

- In the Llama Stack server pod (as environment variables) - to access S3 when creating pipeline runs.
- In the Kubeflow Pipeline pods (via the secret) - to store evaluation results to S3 during pipeline execution.

The LlamaStackDistribution configuration loads these credentials from the "**<ragas_s3_credentials>**" secret and makes them available to both locations.

4. Create a secret for the Kubeflow Pipelines API token:

- a. Get your token by running the following command:

```
$ export KUBEFLOW_PIPELINES_TOKEN=$(oc whoami -t)
```

- b. Create the secret by running the following command:

```
$ oc create secret generic kubeflow-pipelines-token \
  --from-literal=KUBEFLOW_PIPELINES_TOKEN="$KUBEFLOW_PIPELINES_TOKEN"
```

IMPORTANT

The Llama Stack distribution service account does not have privileges to create pipeline runs. This secret provides the necessary authentication token for creating and managing pipeline runs.

5. Verify that the Kubeflow Pipelines endpoint is accessible:

```
$ curl -k -H "Authorization: Bearer $KUBEFLOW_PIPELINES_TOKEN" \
  https://$KUBEFLOW_PIPELINES_ENDPOINT/apis/v1beta1/healthz
```

6. Create a secret for storing your inference model information:

```
$ export INFERENCE_MODEL="llama-3-2-3b"
$ export VLLM_URL="https://llama-32-3b-instruct-predictor:8443/v1"
$ export VLLM_TLS_VERIFY="false" # Use "true" in production
$ export VLLM_API_TOKEN="<token_identifier>"
```

```
$ oc create secret generic llama-stack-inference-model-secret \
  --from-literal INFERENCE_MODEL="$INFERENCE_MODEL" \
  --from-literal VLLM_URL="$VLLM_URL" \
  --from-literal VLLM_TLS_VERIFY="$VLLM_TLS_VERIFY" \
  --from-literal VLLM_API_TOKEN="$VLLM_API_TOKEN"
```

7. Get the KubeFlow Pipelines endpoint by running the following command and searching for "pipeline" in the routes. This is used in a later step for creating a ConfigMap for the Ragas remote provider configuration:

```
$ oc get routes -A | grep -i pipeline
```

This output should show that the namespace, which is the namespace you specified for **KUBEFLOW_NAMESPACE**, has the pipeline server endpoint and the associated metadata one. The one to use is **ds-pipeline-dspa**.

8. Create a ConfigMap for the Ragas remote provider configuration. For example, create a **kubeflow-ragas-config.yaml** file as follows:

Example kubeflow-ragas-config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kubeflow-ragas-config
  namespace: <project_name>
data:
  EMBEDDING_MODEL: "all-MiniLM-L6-v2"
  KUBEFLOW_LLAMA_STACK_URL: "http://$<distribution_name>-
service.$<your_namespace>.svc.cluster.local:$<port>"
  KUBEFLOW_PIPELINES_ENDPOINT: "https://<kfp_endpoint>"
  KUBEFLOW_NAMESPACE: "<project_name>"
  KUBEFLOW_BASE_IMAGE: "quay.io/rhoai/odh-trustyai-ragas-lls-provider-dsp-rhel9:rhoai-
3.0"
  KUBEFLOW_RESULTS_S3_PREFIX: "s3://<bucket_name>/ragas-results"
  KUBEFLOW_S3_CREDENTIALS_SECRET_NAME: "<ragas_s3_credentials>"
```

- **EMBEDDING_MODEL**: Used by Ragas for semantic similarity calculations.
- **KUBEFLOW_LLAMA_STACK_URL**: The URL for the Llama Stack server. This must be accessible from the KubeFlow Pipeline pods. The <distribution_name>, <namespace>, and <port> are replaced with the name of the LlamaStack distribution you are creating, the namespace where you are creating it, and the port. These 3 elements are present in the LlamaStack distribution YAML.
- **KUBEFLOW_PIPELINES_ENDPOINT**: The KubeFlow Pipelines API endpoint URL.
- **KUBEFLOW_NAMESPACE**: The namespace where pipeline runs are executed. This should match your current project namespace.
- **KUBEFLOW_BASE_IMAGE**: The container image used for Ragas evaluation pipeline components. This image contains the Ragas provider package installed via pip.
- **KUBEFLOW_RESULTS_S3_PREFIX**: The S3 path prefix where evaluation results are stored. For example: **s3://my-bucket/ragas-evaluation-results**.

- **KUBEFLOW_S3_CREDENTIALS_SECRET_NAME:** The name of the secret containing S3 credentials.

9. Apply the ConfigMap:

```
$ oc apply -f kubeflow-ragas-config.yaml
```

10. Create a Llama Stack distribution configuration file with the Ragas remote provider. For example, create a llama-stack-ragas-remote.yaml as follows:

Example llama-stack-ragas-remote.yaml

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llama-stack-pod
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: 4
          memory: "12Gi"
        limits:
          cpu: 6
          memory: "14Gi"
    env:
      - name: INFERENCE_MODEL
        valueFrom:
          secretKeyRef:
            key: INFERENCE_MODEL
            name: llama-stack-inference-model-secret
            optional: true
      - name: VLLM_MAX_TOKENS
        value: "4096"
      - name: VLLM_URL
        valueFrom:
          secretKeyRef:
            key: VLLM_URL
            name: llama-stack-inference-model-secret
            optional: true
      - name: VLLM_TLS_VERIFY
        valueFrom:
          secretKeyRef:
            key: VLLM_TLS_VERIFY
            name: llama-stack-inference-model-secret
            optional: true
      - name: VLLM_API_TOKEN
        valueFrom:
          secretKeyRef:
            key: VLLM_API_TOKEN
            name: llama-stack-inference-model-secret
            optional: true
      - name: MILVUS_DB_PATH
        value: ~/milvus.db
```

- name: FMS_ORCHESTRATOR_URL
value: "http://localhost"
- name: KUBEFLOW_PIPELINES_ENDPOINT
valueFrom:
configMapKeyRef:
key: KUBEFLOW_PIPELINES_ENDPOINT
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_NAMESPACE
valueFrom:
configMapKeyRef:
key: KUBEFLOW_NAMESPACE
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_BASE_IMAGE
valueFrom:
configMapKeyRef:
key: KUBEFLOW_BASE_IMAGE
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_LLAMA_STACK_URL
valueFrom:
configMapKeyRef:
key: KUBEFLOW_LLAMA_STACK_URL
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_RESULTS_S3_PREFIX
valueFrom:
configMapKeyRef:
key: KUBEFLOW_RESULTS_S3_PREFIX
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_S3_CREDENTIALS_SECRET_NAME
valueFrom:
configMapKeyRef:
key: KUBEFLOW_S3_CREDENTIALS_SECRET_NAME
name: kubeflow-ragas-config
optional: true
- name: EMBEDDING_MODEL
valueFrom:
configMapKeyRef:
key: EMBEDDING_MODEL
name: kubeflow-ragas-config
optional: true
- name: KUBEFLOW_PIPELINES_TOKEN
valueFrom:
secretKeyRef:
key: KUBEFLOW_PIPELINES_TOKEN
name: kubeflow-pipelines-token
optional: true
- name: AWS_ACCESS_KEY_ID
valueFrom:
secretKeyRef:
key: AWS_ACCESS_KEY_ID
name: "<ragas_s3_credentials>"
optional: true

```

- name: AWS_SECRET_ACCESS_KEY
  valueFrom:
    secretKeyRef:
      key: AWS_SECRET_ACCESS_KEY
      name: "<ragas_s3_credentials>"
      optional: true
- name: AWS_DEFAULT_REGION
  valueFrom:
    secretKeyRef:
      key: AWS_DEFAULT_REGION
      name: "<ragas_s3_credentials>"
      optional: true
name: llama-stack
port: 8321
distribution:
  name: rh-dev

```

11. Deploy the Llama Stack distribution:

```
$ oc apply -f llama-stack-ragas-remote.yaml
```

12. Wait for the deployment to complete:

```
$ oc get pods -w
```

Wait until the **llama-stack-pod** pod status shows **Running**.

3.4. EVALUATING RAG SYSTEM QUALITY WITH RAGAS METRICS

Next steps

Evaluate your RAG system quality by testing your setup, using the example provided in the [demo notebook](#). This demo outlines the basic steps for evaluating your RAG system with Ragas using the Python client. You can execute the demo notebook steps from a Jupyter environment.

Alternatively, you can submit an evaluation by directly using the **http** methods of the [Llama Stack API](#).



IMPORTANT

The Llama Stack pod must be accessible from the Jupyter environment in the cluster, which may not be the case by default. To configure this setup, see [Ingesting content into a Llama model](#)

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- You have created a pipeline server.
- You have created a secret for your AWS credentials in your project namespace.

- You have deployed a Llama Stack distribution with the Ragas evaluation provider enabled (Inline or Remote). For more information, see [Setting up the Ragas inline provider for development](#).
- You have access to a workbench or notebook environment where you can run Python code.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
2. Click the name of the project that contains the workbench.
3. Click the **Workbenches** tab.
4. If the status of the workbench is **Running**, skip to the next step.
If the status of the workbench is **Stopped**, in the **Status** column for the workbench, click **Start**.

The **Status** column changes from **Stopped** to **Starting** when the workbench server is starting, and then to **Running** when the workbench has successfully started.

5. Click the open icon () next to the workbench.
Your Jupyter environment window opens.
6. On the toolbar, click the **Git Clone** icon and then select **Clone a Repository**.
7. In the **Clone a repo** dialog, enter the following URL **https://github.com/trustyai-explainability/llama-stack-provider-ragas.git**
8. In the file browser, select the newly-created **/llama-stack-provider-ragas/demos** folder.
You see a Jupyter notebook named **basic_demo.ipynb**.
9. Double-click the **basic_demo.ipynb** file to launch the Jupyter notebook.
The Jupyter notebook opens. You see code examples for the following tasks:
 - Run your Llama Stack distribution
 - Setup and Imports
 - Llama Stack Client Setup
 - Dataset Preparation
 - Dataset Registration
 - Benchmark Registration
 - Evaluation Execution
 - Inline vs Remote Side-by-side
10. In the Jupyter notebook, run the code cells sequentially through the **Evaluation Execution**.
11. Return to the OpenShift AI dashboard.
12. Click **Develop & train** → **Pipelines** → **Runs**. You might need to refresh the page to see that the new evaluation job running.

13. Wait for the job to show **Successful**.
14. Return to the workbench and run the **Results Display** cell.
15. Inspect the results displayed.

CHAPTER 4. USING LLAMA STACK WITH TRUSTYAI

This section contains tutorials for working with Llama Stack in TrustyAI. These tutorials demonstrate how to use various Llama Stack components and providers to evaluate and work with language models.

The following sections describe how to work with Llama Stack and provide example use cases:

- Using the Llama Stack external evaluation provider with lm-evaluation-harness in TrustyAI
- Running custom evaluations with LM-Eval Llama Stack external evaluation provider
- Using the trustyai-fms Guardrails Orchestrator with Llama Stack

4.1. USING LLAMA STACK EXTERNAL EVALUATION PROVIDER WITH LM-EVALUATION-HARNESS IN TRUSTYAI

This example demonstrates how to evaluate a language model in Red Hat OpenShift AI using the LMEval Llama Stack external eval provider in a Python workbench. To do this, configure a Llama Stack server to use the LMEval eval provider, register a benchmark dataset, and run a benchmark evaluation job on a language model.

Prerequisites

- You have installed Red Hat OpenShift AI, version 2.20 or later.
- You have cluster administrator privileges for your OpenShift AI cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed in your namespace.
- You have installed TrustyAI Operator in your OpenShift AI cluster.
- You have set KServe to Raw Deployment mode in your cluster.

Procedure

1. Create and activate a Python virtual environment for this tutorial in your local machine:

```
python3 -m venv .venv
source .venv/bin/activate
```

2. Install the required packages from the Python Package Index (PyPI):

```
pip install \
  llama-stack \
  llama-stack-client \
  llama-stack-provider-lmeval
```

3. Create the model route:

```
oc create route edge vllm --service=<VLLM_SERVICE> --port=<VLLM_PORT> -n
<MODEL_NAMESPACE>
```

4. Configure the Llama Stack server. Set the variables to configure the runtime endpoint and namespace. The VLLM_URL value should be the **v1/completions** endpoint of your model route and the TRUSTYAI_LM_EVAL_NAMESPACE should be the namespace where your model is deployed. For example:

```
export TRUSTYAI_LM_EVAL_NAMESPACE=<MODEL_NAMESPACE>
export MODEL_ROUTE=$(oc get route -n "$TRUSTYAI_LM_EVAL_NAMESPACE" | awk
'/predictor/{print $2; exit}')
export VLLM_URL="https://${MODEL_ROUTE}/v1/completions"
```

5. Download the **providers.d** provider configuration directory and the **run.yaml** execution file:

```
curl --create-dirs --output providers.d/remote/eval/trustyai_lmeval.yaml
https://raw.githubusercontent.com/trustyai-explainability/llama-stack-provider-
lmeval/refs/heads/main/providers.d/remote/eval/trustyai_lmeval.yaml

curl --create-dirs --output run.yaml https://raw.githubusercontent.com/trustyai-
explainability/llama-stack-provider-lmeval/refs/heads/main/run.yaml
```

6. Start the Llama Stack server in a virtual environment, which uses port **8321** by default:

```
llama stack run run.yaml --image-type venv
```

7. Create a Python script in a Jupyter workbench and import the following libraries and modules, to interact with the server and run an evaluation:

```
import os
import subprocess

import logging

import time
import pprint
```

8. Start the Llama Stack Python client to interact with the running Llama Stack server:

```
BASE_URL = "http://localhost:8321"

def create_http_client():
    from llama_stack_client import LlamaStackClient
    return LlamaStackClient(base_url=BASE_URL)

client = create_http_client()
```

9. Print a list of the current available benchmarks:

```

benchmarks = client.benchmarks.list()

pprint.pprint(f"Available benchmarks: {benchmarks}")

```

10. LMEval provides access to over 100 preconfigured evaluation datasets. Register the ARC-Easy benchmark, a dataset of grade-school level, multiple-choice science questions:

```

client.benchmarks.register(
    benchmark_id="trustyai_lmeval::arc_easy",
    dataset_id="trustyai_lmeval::arc_easy",
    scoring_functions=["string"],
    provider_benchmark_id="string",
    provider_id="trustyai_lmeval",
    metadata={
        "tokenizer": "google/flan-t5-small",
        "tokenized_requests": False,
    }
)

```

11. Verify that the benchmark has been registered successfully:

```

benchmarks = client.benchmarks.list()
pprint.print(f"Available benchmarks: {benchmarks}")

```

12. Run a benchmark evaluation job on your deployed model using the following input. Replace phi-3 with the name of your deployed model:

```

job = client.eval.run_eval(
    benchmark_id="trustyai_lmeval::arc_easy",
    benchmark_config={
        "eval_candidate": {
            "type": "model",
            "model": "phi-3",
            "provider_id": "trustyai_lmeval",
            "sampling_params": {
                "temperature": 0.7,
                "top_p": 0.9,
                "max_tokens": 256
            },
        },
        "num_examples": 1000,
    },
)

print(f"Starting job '{job.job_id}'")

```

13. Monitor the status of the evaluation job using the following code. The job will run asynchronously, so you can check its status periodically:

```

def get_job_status(job_id, benchmark_id):
    return client.eval.jobs.status(job_id=job_id, benchmark_id=benchmark_id)

while True:
    job = get_job_status(job_id=job.job_id, benchmark_id="trustyai_lmeval::arc_easy")

```

```
print(job)

if job.status in ['failed', 'completed']:
    print(f"Job ended with status: {job.status}")
    break

time.sleep(20)
```

1. Retrieve the evaluation job results once the job status reports back as **completed**:

```
pprint.pprint(client.eval.jobs.retrieve(job_id=job.job_id,
benchmark_id="trustyai_lmeval::arc_easy").scores)
```

4.2. RUNNING CUSTOM EVALUATIONS WITH LM-EVAL AND LLAMA STACK

This example demonstrates how to use the [LM-Eval Llama Stack external eval provider](#) to evaluate a language model with a custom benchmark. Creating a custom benchmark is useful for evaluating specific model knowledge and behavior.

The process involves three steps:

- Uploading the task dataset to your OpenShift AI cluster
- Registering it as a custom benchmark dataset with Llama Stack
- Running a benchmark evaluation job on a language model

Prerequisites

- You have installed Red Hat OpenShift AI, version 2.20 or later.
- You have cluster administrator privileges for your OpenShift AI cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed on vLLM Serving Runtime in your OpenShift AI cluster.
- You have installed TrustyAI Operator in your OpenShift AI cluster.
- You have set KServe to Raw Deployment mode in your cluster.

Procedure

1. Upload your custom benchmark dataset to your OpenShift cluster using a PersistentVolumeClaim (PVC) and a temporary pod. Create a PVC named **my-pvc** to store your dataset. Run the following command in your CLI, replacing <MODEL_NAMESPACE> with the namespace of your language model:

```
oc apply -n <MODEL_NAMESPACE> -f - << EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
EOF
```

2. Create a pod object named **dataset-storage-pod** to download the task dataset into the PVC. This pod is used to copy your dataset from your local machine to the OpenShift AI cluster:

```
oc apply -n <MODEL_NAMESPACE> -f - << EOF
apiVersion: v1
kind: Pod
metadata:
  name: dataset-storage-pod
spec:
  containers:
    - name: dataset-container
      image: 'quay.io/prometheus/busybox:latest'
      command: ["/bin/sh", "-c", "sleep 3600"]
      volumeMounts:
        - mountPath: "/data/upload_files"
          name: dataset-storage
  volumes:
    - name: dataset-storage
      persistentVolumeClaim:
        claimName: my-pvc
EOF
```

3. Copy your locally stored task dataset to the pod to place it within the PVC. In this example, the dataset is named **example-dk-bench-input-bmo.jsonl** locally and it is copied to the **dataset-storage-pod** under the path **/data/upload_files/**.

```
oc cp example-dk-bench-input-bmo.jsonl dataset-storage-pod:/data/upload_files/example-dk-bench-input-bmo.jsonl -n <MODEL_NAMESPACE>
```

4. Once the custom dataset is uploaded to the PVC, register it as a benchmark for evaluations. At a minimum, provide the following metadata and replace the **DK_BENCH_DATASET_PATH** and any other metadata fields to match your specific configuration:
 - a. The TrustyAI LM-Eval Tasks GitHub web address
 - b. Your branch
 - c. The commit hash and path of the custom task.

```
client.benchmarks.register(
  benchmark_id="trustyai_lmeval::dk-bench",
  dataset_id="trustyai_lmeval::dk-bench",
```

```

    scoring_functions=["accuracy"],
    provider_benchmark_id="dk-bench",
    provider_id="trustyai_lmeval",
    metadata={
        "custom_task": {
            "git": {
                "url": "https://github.com/trustyai-explainability/lm-eval-tasks.git",
                "branch": "main",
                "commit": "8220e2d73c187471acbe71659c98bccecf77958",
                "path": "tasks/",
            }
        },
        "env": {
            # Path of the dataset inside the PVC
            "DK_BENCH_DATASET_PATH": "/opt/app-root/src/hf_home/example-dk-bench-
input-bmo.jsonl",
            "JUDGE_MODEL_URL": "http://phi-3-predictor:8080/v1/chat/completions",
            # For simplicity, we use the same model as the one being evaluated
            "JUDGE_MODEL_NAME": "phi-3",
            "JUDGE_API_KEY": "",
        },
        "tokenized_requests": False,
        "tokenizer": "google/flan-t5-small",
        "input": {"storage": {"pvc": "my-pvc"}}
    },
)

```

5. Run a benchmark evaluation on your model:

```

job = client.eval.run_eval(
    benchmark_id="trustyai_lmeval::dk-bench",
    benchmark_config={
        "eval_candidate": {
            "type": "model",
            "model": "phi-3",
            "provider_id": "trustyai_lmeval",
            "sampling_params": {
                "temperature": 0.7,
                "top_p": 0.9,
                "max_tokens": 256
            },
        },
        "num_examples": 1000,
    },
)

print(f"Starting job '{job.job_id}'")

```

6. Monitor the status of the evaluation job. The job runs asynchronously, so you can check its status periodically:

```

import time
def get_job_status(job_id, benchmark_id):
    return client.eval.jobs.status(job_id=job_id, benchmark_id=benchmark_id)

while True:

```

```

job = get_job_status(job_id=job.job_id, benchmark_id="trustyai_lmeval::dk-bench")
print(job)

if job.status in ['failed', 'completed']:
    print(f"Job ended with status: {job.status}")
    break

time.sleep(20)

```

4.3. DETECTING PERSONALLY IDENTIFIABLE INFORMATION (PII) BY USING GUARDRAILS WITH LLAMA STACK

The **trustyai_fms** Orchestrator server is an external provider for Llama Stack that allows you to configure and use the Guardrails Orchestrator and compatible detection models through the Llama Stack API. This implementation of Llama Stack combines [Guardrails Orchestrator](#) with a suite of community-developed detectors to provide robust content filtering and safety monitoring.

This example demonstrates how to use the built-in [Guardrails Regex Detector](#) to detect personally identifiable information (PII) with Guardrails Orchestrator as Llama Stack safety guardrails, using the **LlamaStack** Operator to deploy a distribution in your Red Hat OpenShift AI namespace.



NOTE

Guardrails Orchestrator with Llama Stack is not supported on **s390x**, as it requires the LlamaStack Operator, which is currently unavailable for this architecture.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed in your namespace.
- A cluster administrator has installed the following Operators in OpenShift:
 - Red Hat Authorino Operator, version 1.2.1 or later
 - Red Hat OpenShift Service Mesh, version 2.6.7-0 or later

Procedure

1. Configure your OpenShift AI environment with the following configurations in the **DataScienceCluster**. Note that you must manually update the **spec.llamastack.managementState** field to **Managed**:

```

spec:
  trustyai:

```

```

managementState: Managed
llamastack:
  managementState: Managed
kserve:
  defaultDeploymentMode: RawDeployment
  managementState: Managed
nim:
  managementState: Managed
  rawDeploymentServiceConfig: Headless
serving:
  ingressGateway:
    certificate:
      type: OpenshiftDefaultIngress
    managementState: Removed
    name: knative-serving
  serviceMesh:
    managementState: Removed

```

2. Create a project in your OpenShift AI namespace:

```

PROJECT_NAME="lls-minimal-example"
oc new-project $PROJECT_NAME

```

3. Deploy the Guardrails Orchestrator with regex detectors by applying the Orchestrator configuration for regex-based PII detection:

```

cat <<EOF | oc apply -f -
kind: ConfigMap
apiVersion: v1
metadata:
  name: fms-orchestr8-config-nlp
data:
  config.yaml: |
    detectors:
      regex:
        type: text_contents
        service:
          hostname: "127.0.0.1"
          port: 8080
          chunker_id: whole_doc_chunker
          default_threshold: 0.5
---
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: GuardrailsOrchestrator
metadata:
  name: guardrails-orchestrator
spec:
  orchestratorConfig: "fms-orchestr8-config-nlp"
  enableBuiltInDetectors: true
  enableGuardrailsGateway: false
  replicas: 1
EOF

```

4. In the same namespace, create a Llama Stack distribution:


```

apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llamastackdistribution-sample
  namespace: <PROJECT_NAMESPACE>
spec:
  replicas: 1
  server:
    containerSpec:
      env:
        - name: VLLM_URL
          value: '${VLLM_URL}'
        - name: INFERENCE_MODEL
          value: '${INFERENCE_MODEL}'
        - name: MILVUS_DB_PATH
          value: '~/llama/milvus.db'
        - name: VLLM_TLS_VERIFY
          value: 'false'
        - name: FMS_ORCHESTRATOR_URL
          value: '${FMS_ORCHESTRATOR_URL}'
      name: llama-stack
      port: 8321
    distribution:
      name: rh-dev
  storage:
    size: 20Gi

```



NOTE

— After deploying the **LlamaStackDistribution** CR, a new pod is created in the same namespace. This pod runs the LlamaStack server for your distribution. —

5. Once the Llama Stack server is running, use the **/v1/shields** endpoint to dynamically register a shield. For example, register a shield that uses regex patterns to detect personally identifiable information (PII).
6. Open a port-forward to access it locally:

```
oc -n $PROJECT_NAME port-forward svc/llama-stack 8321:8321
```

7. Use the **/v1/shields** endpoint to dynamically register a shield. For example, register a shield that uses regex patterns to detect personally identifiable information (PII):

```

curl -X POST http://localhost:8321/v1/shields \
-H 'Content-Type: application/json' \
-d '{
  "shield_id": "regex_detector",
  "provider_shield_id": "regex_detector",
  "provider_id": "trustyai_fms",
  "params": {
    "type": "content",
    "confidence_threshold": 0.5,
    "message_types": ["system", "user"],
    "detectors": {

```

```

    "regex": {
      "detector_params": {
        "regex": ["email", "us-social-security-number", "credit-card"]
      }
    }
  }
}
}'

```

8. Verify that the shield was registered:

```
curl -s http://localhost:8321/v1/shields | jq .'
```

9. The following output indicates that the shield has been registered successfully:

```

{
  "data": [
    {
      "identifier": "regex_detector",
      "provider_resource_id": "regex_detector",
      "provider_id": "trustyai_fms",
      "type": "shield",
      "params": {
        "type": "content",
        "confidence_threshold": 0.5,
        "message_types": [
          "system",
          "user"
        ],
        "detectors": {
          "regex": {
            "detector_params": {
              "regex": [
                "email",
                "us-social-security-number",
                "credit-card"
              ]
            }
          }
        }
      }
    }
  ]
}

```

10. Once the shield has been registered, verify that it is working by sending a message containing PII to the **/v1/safety/run-shield** endpoint:
 - a. Email detection example:

```

curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
  "shield_id": "regex_detector",
  "messages": [

```

```
{
  "content": "My email is test@example.com",
  "role": "user"
}
]
}' | jq .'
```

This should return a response indicating that the email was detected:

```
{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence: 1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
      }
    },
    "results": [
      {
        "message_index": 0,
        "text": "My email is test@example.com",
        "status": "violation",
        "score": 1.0,
        "detection_type": "pii",
        "individual_detector_results": [
          {
            "detector_id": "regex",
            "status": "violation",
            "score": 1.0,
            "detection_type": "pii"
          }
        ]
      }
    ]
  }
}
```

b. Social security number (SSN) detection example:

```
curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
  "shield_id": "regex_detector",
  "messages": [
    {
      "content": "My SSN is 123-45-6789",
      "role": "user"
    }
  ]
}' | jq .'
```

This should return a response indicating that the SSN was detected:

```
{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence: 1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
      }
    },
    "results": [
      {
        "message_index": 0,
        "text": "My SSN is 123-45-6789",
        "status": "violation",
        "score": 1.0,
        "detection_type": "pii",
        "individual_detector_results": [
          {
            "detector_id": "regex",
            "status": "violation",
            "score": 1.0,
            "detection_type": "pii"
          }
        ]
      }
    ]
  }
}
```

```

    }
  }
}

```

c. Credit card detection example:

```

curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
  "shield_id": "regex_detector",
  "messages": [
    {
      "content": "My credit card number is 4111-1111-1111-1111",
      "role": "user"
    }
  ]
}' | jq '.'

```

This should return a response indicating that the credit card number was detected:

```

{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence: 1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
      }
    }
  },
  "results": [
    {
      "message_index": 0,
      "text": "My credit card number is 4111-1111-1111-1111",
      "status": "violation",
      "score": 1.0,
      "detection_type": "pii",
      "individual_detector_results": [
        {
          "detector_id": "regex",

```

```
    "status": "violation",  
    "score": 1.0,  
    "detection_type": "pii"  
  }  
]  
}  
]  
}  
}  
}
```